学号：2022013106

西北农林科技大学
NORTHWEST A&F UNIVERSITY

2022 级本科生

图像处理与机器视觉课程论文

基于YOLOv8的猕猴桃果实图像识别系统设计

学 院（系）：机械与电子工程学院

专　　业：农业机械化及其自动化

年 级 班 级：机化2201

学 生 姓 名：黄钊伟

完 成 日 期：2024年12月2日

基于 YOLOv8 的猕猴桃果实图像识别系统设计

摘 要：

随着人工智能技术的快速发展，计算机视觉技术在农业领域得到了广泛的应用，特别是在农产品的自动化分拣、品质检测等方面。猕猴桃作为营养价值高且受欢迎的水果之一，其采摘和分拣工作往往依赖于人工，效率低下且成本较高。

为了提高猕猴桃采摘和分拣的效率和准确性，本文提出了基于 YOLOv8 算法的猕猴桃果实识别系统。YOLOv8 作为最新的目标检测算法，以其快速、准确的检测能力在图像识别领域受到广泛关注。本文首先介绍了 YOLOv8 算法的背景和应用前景，随后详细阐述了 Ultralytics 环境的配置过程，包括 Miniconda 的安装、PyPI 的配置以及 PyTorch 的安装。在数据集准备阶段，本文从 Roboflow 网站获取了猕猴桃果实数据集，并使用 labelImg 和 Make Sense 工具进行了数据标注。模型训练与部署章节中，本文详细描述了模型训练的配置、训练过程以及模型的评估和部署。通过实验验证，本文所提出的系统在验证集上的识别成功率达到了 100%，展现了极高的识别准确率。此外，本文还设计了一个人机交互 UI 软件界面，使用 Python 软件实现了对输入图片中猕猴桃果实的检测和标注，为本人科创项目猕猴桃采摘机械臂设计提供视觉识别模块的设计思路与基础。

关键词：计算机视觉;猕猴桃识别;YOLOv8;视觉模块

目　录

第 1 章 绪论 1
1.1 选题的目的和意义 1
1.2 主要设计内容 1
第 2 章 YOLOv8 算法环境配置 2
2.1 Ultralytics 介绍 2
2.2 Miniconda 下载与安装 2
2.3 PyPI 的配置下载 3
2.4 Pytorch 的配置安装 4
第 3 章 数据集准备 5
3.1 模型训练用数据集 5
3.2 验证用数据集 5
3.3 数据标注 6
第 4 章 模型训练与部署 8
4.1 模型训练配置 8
4.2 模型评估 9
4.3 模型部署与预测 10
第 5 章 人机交互软件设计 11
5.1 UI 界面设计 11
5.2 APP 测试 12
第 6 章 总结展望 14
6.1 总结 14
6.2 展望 14
致谢 16
参考文献 17
附　录 18

第 1 章 绪论

1.1 选题的目的和意义

在 21 世纪的今天，随着全球人口的增长和生活水平的提高，对农产品的需求量不断增加，尤其是对营养价值高、口感独特的猕猴桃等特色水果的需求日益增长[1]。猕猴桃作为一种富含维生素 C 和其他营养成分的水果，在全球范围内享有盛誉。然而，传统的猕猴桃采摘和分拣工作主要依赖于人工，这不仅效率低下，而且成本高昂，特别是在劳动力成本不断上升的背景下，这一问题变得尤为突出[2]。此外，人工操作的准确性和一致性难以保证，这对猕猴桃的品质控制和市场竞争力构成了挑战[3]。

随着人工智能技术的快速发展，计算机视觉技术在农业领域的应用越来越广泛，特别是在水果采摘、分拣和品质检测等方面[4]。在猕猴桃识别领域，计算机视觉技术的应用可以极大地提高分拣的自动化水平，减少人工成本，提升作业效率。深度学习作为计算机视觉的一个重要分支，通过构建复杂的神经网络模型，使得机器能够自动学习图像中的特征，极大地提高了识别的准确性[5]。YOLO（You Only Look Once）系列算法作为深度学习领域中的目标检测算法，以其快速、准确的检测能力而受到广泛关注。YOLOv8 作为该系列的最新版本，进一步优化了检测速度和准确率，使其在工业和农业自动化领域具有广泛的应用前景。

Ultralytics 公司作为 YOLO 算法的主要开发者之一，提供了一套完整的 YOLOv8 算法实现和工具链，极大地降低了算法使用和集成的难度。Ultralytics 环境的易用性和高效性，使得研究人员和开发者能够快速地将 YOLOv8 算法应用于实际问题中。本文将基于 Ultralytics 提供的 YOLOv8 算法，设计并实现一个猕猴桃识别系统，以期为猕猴桃的自动化采摘和品质控制提供技术支持。本文设计该系统的主要目的是为作者的大创项目，猕猴桃采摘机械臂设计提供视觉模块基础，也期待后续能进一步完善系统。

1.2 主要设计内容

本文基于 YOLOv8 算法，设计猕猴桃果实识别系统，具备适用范围广、识别准确率高的特点。为实现 YOLOv8 算法的运行，配置了用于运行 YOLO 算法的 Ultralytics 环境，为实现模型训练，使用了开源数据集，此外自行准备了用于验证数据集的图片，从而降低偶然误差，为实现模型部署应用，基于 Python 软件设计了 UI 界面实现人机交互，完成对输入图片中猕猴桃果实的检测。概括主要设计内容如下：

（1）算法运行环境配置（Ultralytics 环境）；

（2）准备用于模型训练的数据集以及用于验证的图片；

（3）最优模型训练策略部署；

（4）人机交互 UI 程序界面设计；

第 2 章 YOLOv8 算法环境配置

在深度学习和计算机视觉领域，一个稳定且高效的开发环境对于算法的研究和应用至关重要。Ultralytics 环境配置是实现 YOLOv8 猕猴桃识别系统的基础，涉及到 Ultralytics 框架介绍、Miniconda 的下载与安装、PyPI 的配置以及 PyTorch 的下载与安装。

2.1 Ultralytics 介绍

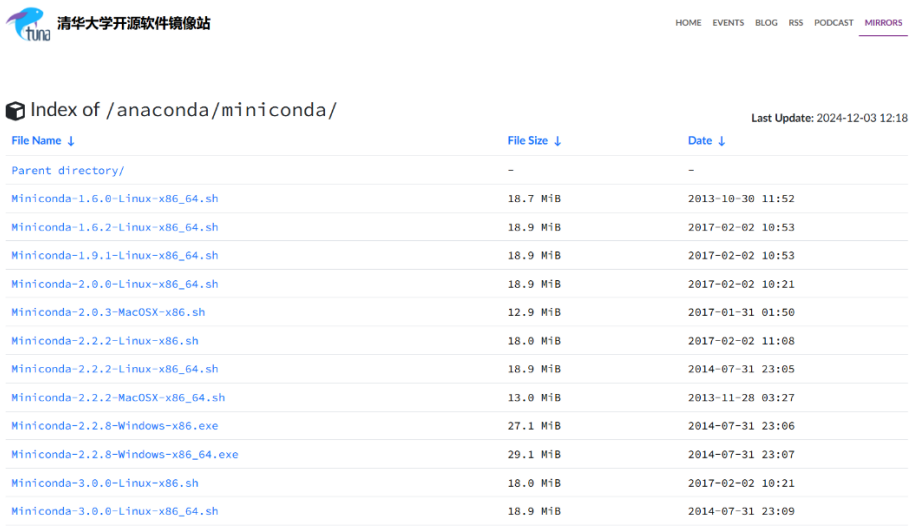
Ultralytics 是一个专注于深度学习目标检测算法的公司，以其开发的 YOLO（You Only Look Once）系列算法而闻名。YOLO 算法以其快速、准确的目标检测能力在业界受到广泛认可，尤其是在需要实时处理的场景中。Ultralytics 提供的 YOLOv8 算法是该系列的最新版本，它在保持高速度的同时，进一步提升了检测的准确性。Ultralytics 还提供了易于使用的软件框架，使得研究人员和开发者能够快速地将 YOLOv8 算法集成到他们的项目中。

图 2-1 Ultralytics

2.2 Miniconda 下载与安装

Miniconda 是一个轻量级的 Python 发行版，包含了 conda 包管理器，允许用户轻松地安装、运行和管理多个 Python 环境和包。对于深度学习项目，Miniconda 提供了一个干净的环境，可以避免不同项目之间的依赖冲突。

Miniconda 的下载是在清华大学开源软件镜像站中完成的，安装的版本为“Miniconda3-py38_22.11.1-1-Windows-x86_64.exe”。



清华大学开源软件镜像站

HOME EVENTS BLOG RSS PODCAST MIRRORS

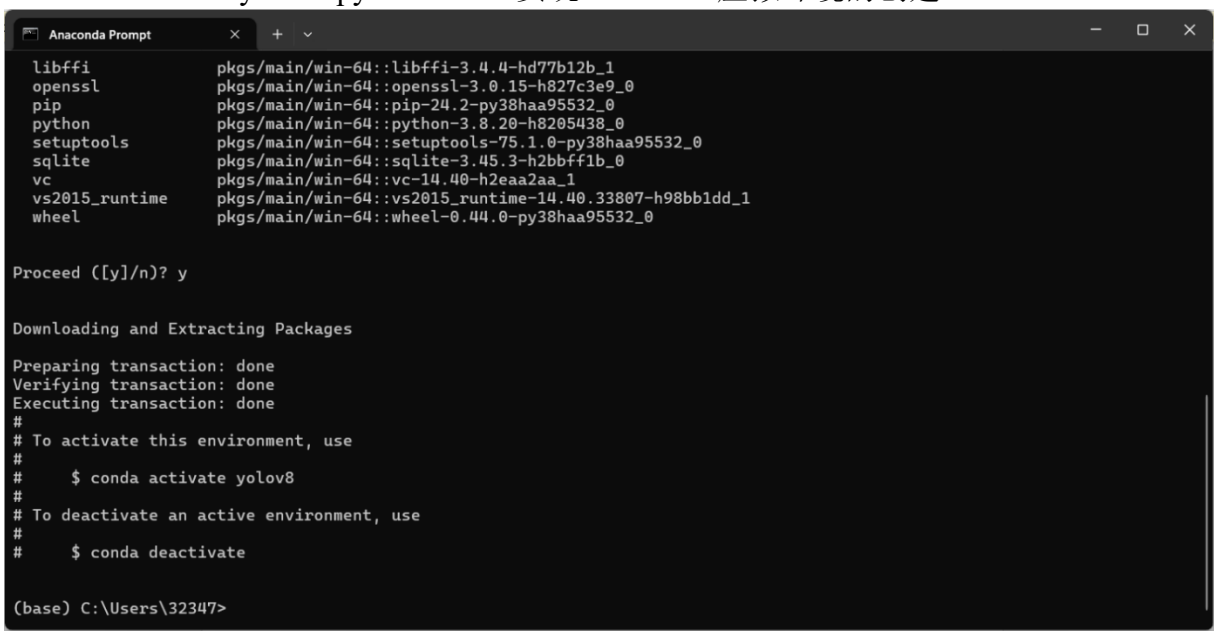
Index of /anaconda/miniconda/

Last Update: 2024-12-03 12:18

File Name ↓	File Size ↓	Date ↓
Parent directory/	-	-
Miniconda-1.6.0-Linux-x86_64.sh	18.7 MiB	2013-10-30 11:52
Miniconda-1.6.2-Linux-x86_64.sh	18.9 MiB	2017-02-02 10:53
Miniconda-1.9.1-Linux-x86_64.sh	18.9 MiB	2017-02-02 10:53
Miniconda-2.0.0-Linux-x86_64.sh	18.9 MiB	2017-02-02 10:21
Miniconda-2.0.3-MacOSX-x86.sh	12.9 MiB	2017-01-31 01:50
Miniconda-2.2.2-Linux-x86.sh	18.0 MiB	2017-02-02 11:08
Miniconda-2.2.2-Linux-x86_64.sh	18.9 MiB	2014-07-31 23:05
Miniconda-2.2.2-MacOSX-x86_64.sh	13.0 MiB	2013-11-28 03:27
Miniconda-2.2.8-Windows-x86.exe	27.1 MiB	2014-07-31 23:06
Miniconda-2.2.8-Windows-x86_64.exe	29.1 MiB	2014-07-31 23:07
Miniconda-3.0.0-Linux-x86.sh	18.0 MiB	2017-02-02 10:21
Miniconda-3.0.0-Linux-x86_64.sh	18.9 MiB	2014-07-31 23:09

图 2-2 miniconda 下载

安装完成后需要用 Miniconda 实现虚拟环境的配置，在 Anaconda Prompt 中输入“conda create -n yolov8 python=3.8”实现 YOLOv8 虚拟环境的创建。



```
Anaconda Prompt
  libffi             pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
  openssl            pkgs/main/win-64::openssl-3.0.15-h827c3e9_0
  pip                pkgs/main/win-64::pip-24.2-py38haa95532_0
  python             pkgs/main/win-64::python-3.8.20-h8205438_0
  setuptools         pkgs/main/win-64::setuptools-75.1.0-py38haa95532_0
  sqlite             pkgs/main/win-64::sqlite-3.45.3-h2bbff1b_0
  vc                 pkgs/main/win-64::vc-14.40-h2eaa2aa_1
  vs2015_runtime     pkgs/main/win-64::vs2015_runtime-14.40.33807-h98bb1dd_1
  wheel              pkgs/main/win-64::wheel-0.44.0-py38haa95532_0

Proceed ([y]/n)? y


Downloading and Extracting Packages

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate yolov8
#
# To deactivate an active environment, use
#
#     $ conda deactivate


(base) C:\Users\32347>
```

图 2-3 虚拟环境创建

2.3 PyPI 的配置下载

PyPI（Python Package Index）是 Python 的官方第三方软件仓库，提供了大量的 Python 包。在使用 conda 创建新的 Python 环境后，通常需要配置 PyPI，以便能够安装和管理 Python 包。

PyPI 下载同样使用清华大学开源软件镜像站完成，在创建的 YOLOv8 虚拟环境中使用“pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple”

命令完成源的配置。

图 2-4 PyPI 下载安装

2.4 Pytorch 的配置安装

PyTorch 是一个开源的机器学习库，广泛用于计算机视觉和自然语言处理领域。它提供了强大的 GPU 加速的张量计算能力，并且与 YOLOv8 算法兼容，也是 Ultralytics 官方推荐使用的深度学习框架。

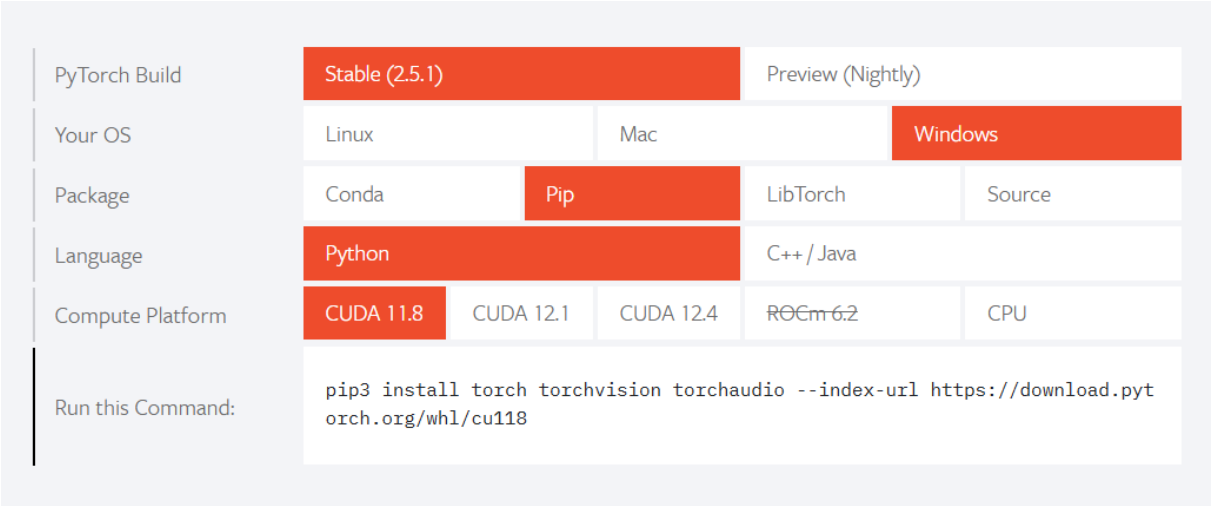


图 2-5 Pytorch 安装

第 3 章 数据集准备

数据集的质量对于机器学习和深度学习项目的成功至关重要，尤其是在目标检测领域。一个丰富、多样化且标注准确的数据集能够显著提高模型的性能和泛化能力。本章将详细介绍猕猴桃识别系统中数据集的准备工作，包括模型训练用数据集获取、验证用数据集的准备、预处理以及数据标注等关键步骤。

3.1 模型训练用数据集

由于训练一个可靠模型的过程中所需的数据集非常庞大，手动拍摄或者网络图片下载都是非常大的工作量，因此本文选择从开源的数据集网站上获取猕猴桃果实的相关数据集，来提高模型训练的效率。

国内常用的开源数据集网站有百度飞桨、阿里云天池等，国外可用的开源数据集网站则有 Github，OpenDataLab 以及 Roboflow 等，本文最终选择从 Roboflow 网站中获取猕猴桃果实数据集。

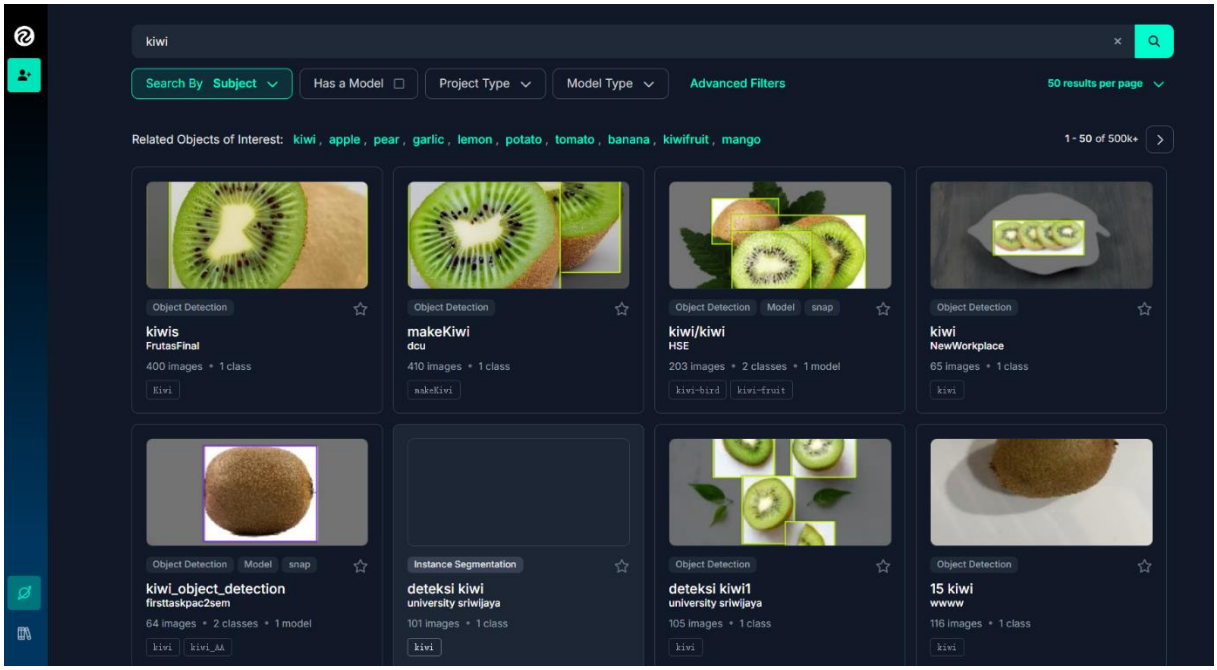


图 3-1 Roboflow 的猕猴桃果实数据集

3.2 验证用数据集

虽然在模型训练用数据集中划分时已经按 7：3 的比例区分了训练集与验证集，但仍旧需要准备几张图片来进行最终训练出的模型的验证。对验证用数据集中的图片要求相对不高，与模型训练用数据集中的图片不能出现类似或重复，并且要有一定识别难度来检测模型对猕猴桃果实识别的成功率，当然，用于验证的也不只局限于图片，由于 YOLOv8 训练出的模型在进行模型预测时可以在 source 中调用视频，因此视频也可以作

为模型验证的重要指标凭证。

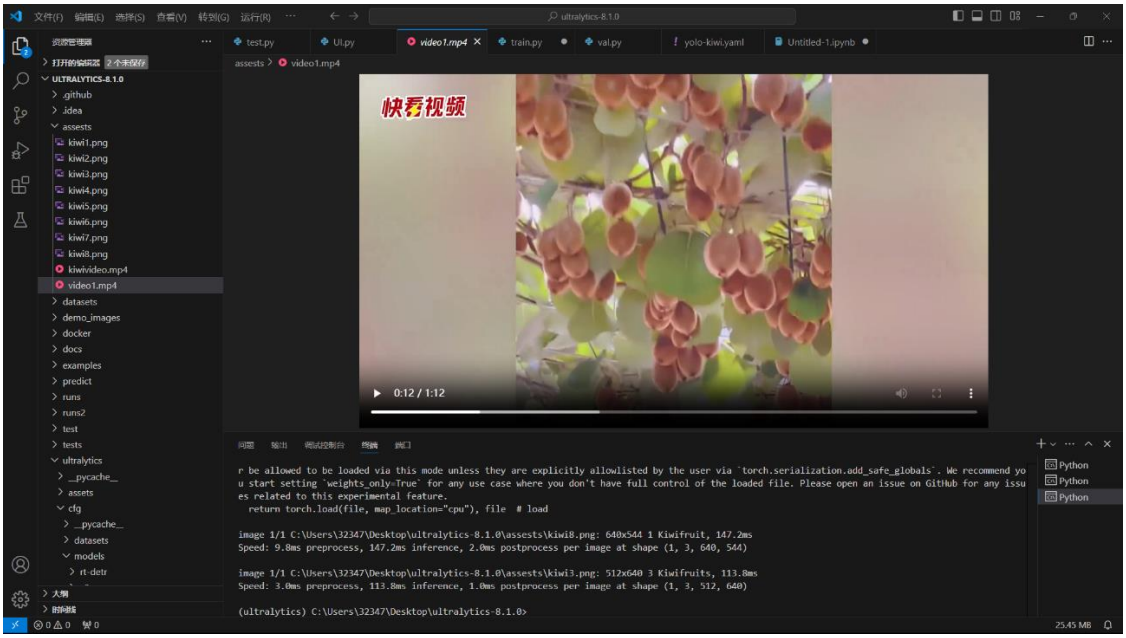


图 3-2 模型验证视频

3.3 数据标注

数据标注是数据集准备过程中最繁琐的步骤，理论上来说在开源网站上直接下载的数据集是无需标注的，而本文自行建立的验证用数据集也只是用来检测识别成功率，因此也没有必要标注。但是，为了说明本文没有偷工减料，用已有的数据集偷懒，最终还是选择对自建验证用数据集进行数据标注。

数据标注可用的网站或工具也有很多，在 Python 中使用较为方便的就是库中配置的 labelImg 小插件，可批量插入图片然后进行标注，本文的验证用数据集采取的标注方式正是这种。

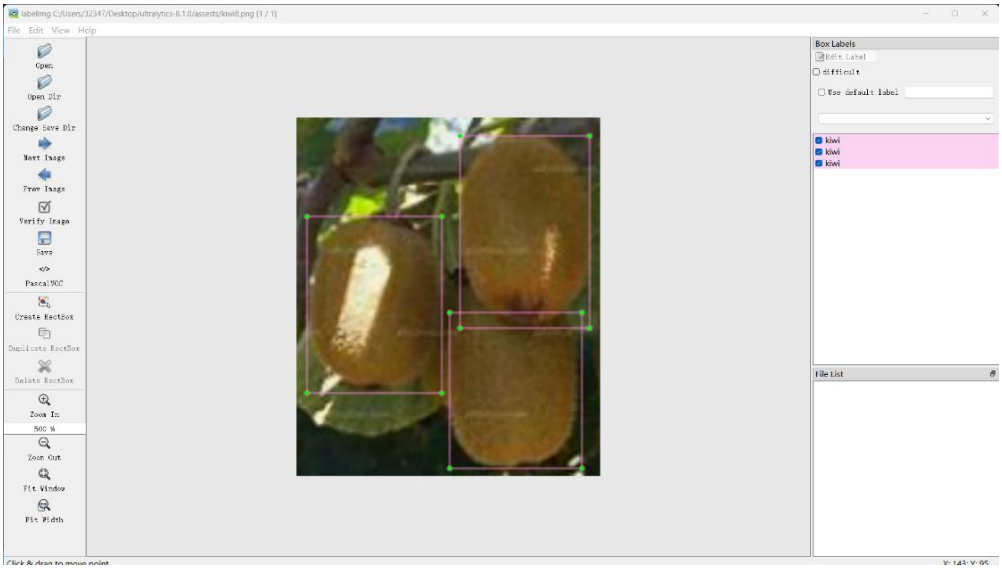


图 3-3 labelImg 数据标注

但是 labelImg 在数据集较大时会因操作繁琐而捉襟见肘，这时就可以采用在线数据标注网站 Make Sense 进行标注。该网站上不仅标注方便快捷，而且可以在标注结束后直接将所有图片打包输出为 YOLO 训练模型用的对应模式，大大减少了数据集中数据和标签对应的工作量，并且所生成的标签也可以直接使用。

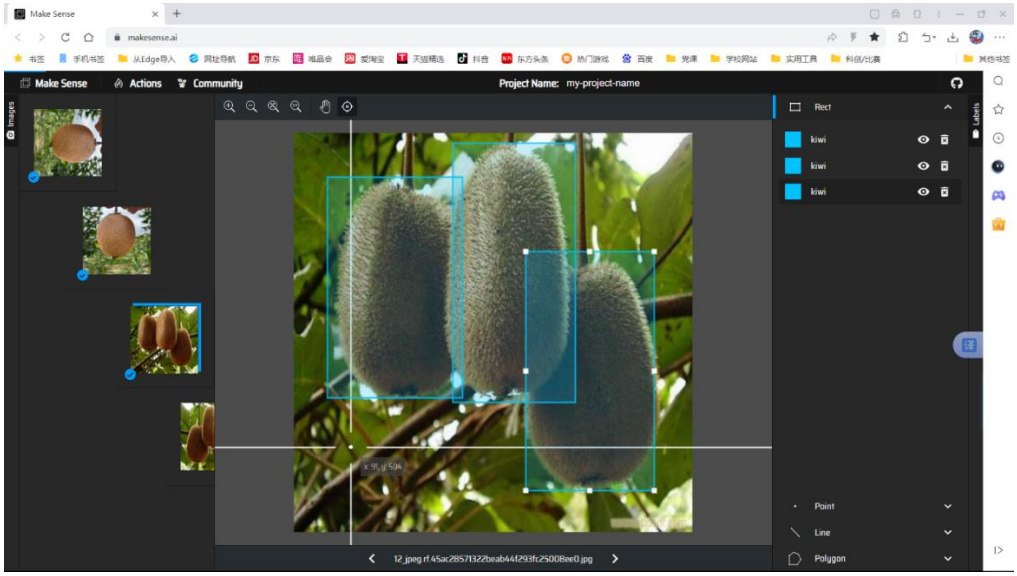


图 3-4 Make Sense 数据标注

第 4 章 模型训练与部署

在深度学习项目中，模型的训练与部署是实现算法性能的关键步骤。本章将详细描述基于 Ultralytics 框架的 YOLOv8 模型的训练过程和部署策略，包括模型训练配置、训练过程、模型评估以及最终的部署。

4.1 模型训练配置

在准备好模型训练用数据集后，便准备进行模型训练，首先进行的是 yaml 格式的参数文件配置，主要是对数据集的路径以及训练集、验证集文件夹的标注，还有测试集可选择，但本文没有设置，后续在大创项目实施时会根据现存的不足进行改进。

```
! yolo-kiwi.yaml
1   # Ultralytics YOLO 🚀, AGPL-3.0 license
2   # COCO8 dataset (first 8 images from COCO train2017) by Ultralytics
3   # Documentation: https://docs.ultralytics.com/datasets/detect/coco8/
4   # Example usage: yolo train data=coco8.yaml
5   # parent
6   # ├── ultralytics
7   # └── datasets
8   #     └── coco8  ← downloads here (1 MB)
9
10
11  # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]
12  path: kiwi  # dataset root dir
13  train: images/train  # train images (relative to 'path') 4 images
14  val: images/val  # val images (relative to 'path') 4 images
15  test:  # test images (optional)
16
17  # Classes
18  names:
19    0: Kiwifruit
20
```

图 4-1 yaml 参数文件配置

配置好参数文件后，开始进行模型训练，这里有两种办法开始模型训练，一种是通过编写 train.py 也就是训练的脚本程序来开始模型训练；另一种则是用本文在第二章中完成的虚拟环境来进行，也就是在终端中输入命令开始训练，使用的指令为 yolo task=detect mode=train model=./yolov8n.pt data =yolo-kiwi.yaml epochs=200 workers=1 batch=16。本文采用的是终端运行指令，其中最重要的就是训练轮次的设置，为了保证模型的识别效果和使用效率，本文将训练轮次设置为 200 轮。

```
train.py > ...
1   from ultralytics import YOLO
2
3   model = YOLO('yolov8m.pt')
4
5   model.train(data='yolo-kiwi.yaml',workers=1,epochs=200,batch=16)
6
```

图 4-2 train.py 训练程序编写

4.2 模型评估

模型训练完成后，需要对所训练出的模型进行评估，这里主要参照两个指标，一个是模型对预设好的验证集的识别效果，较为直观，另一个是模型的精度置信度曲线，反映的是模型训练了 200 轮后识别效果的置信度，相对更具有统计学意义。

从模型对验证集的识别效果中可以看出模型对直接摆放的猕猴桃果实的识别成功率达到 100%，但对在果树上的猕猴桃果实识别有时会因光照或果实重叠导致出现偏差，但总体上看识别效果已经达到了预期要求。

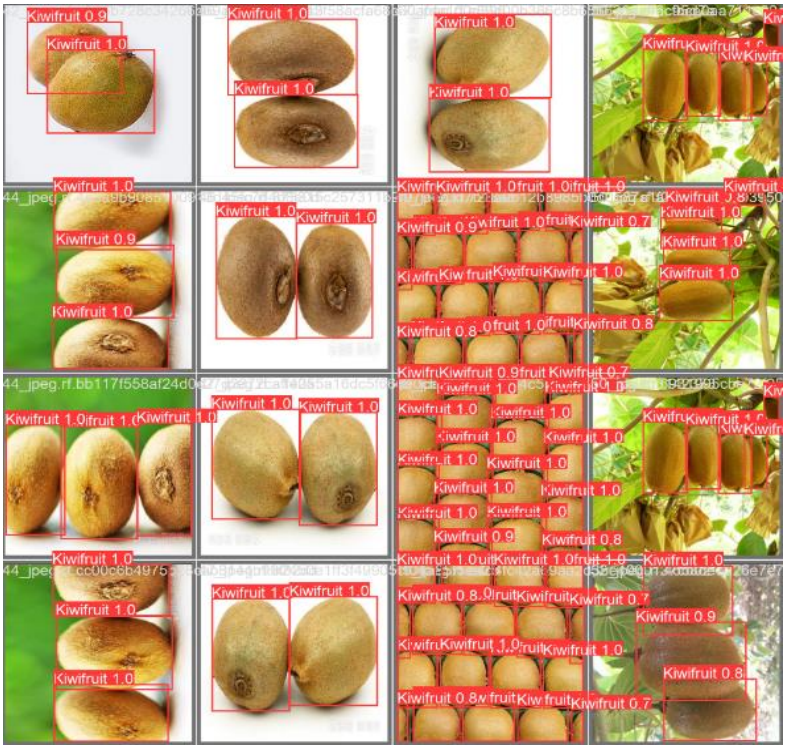


图 4-3 验证集识别效果

从模型的精度置信度曲线中不难看出随着训练轮次的增加，置信度与精度均呈快速上升趋势，最终训练完成后，两者都近似为 1，符合目前对模型的使用要求，可以运用在我的大创项目中，实现对猕猴桃果实的识别，但后续使用中出现问题仍然需要对现有模型进行优化。

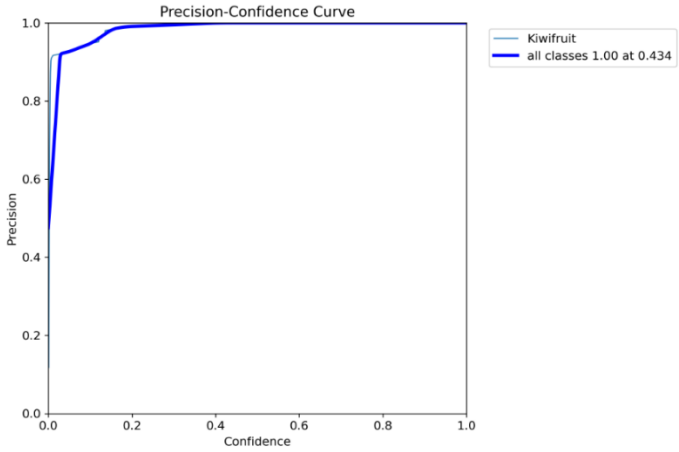


图 4-4 精度置信度曲线

4.3 模型部署与预测

在评估完成确认了模型的识别准确率后，下一步是对模型进行部署，这里编写了 test.py 的程序来实现对模型的部署，使用了从网络上下载的一张单个猕猴桃果实的图片，最终用 209ms 完成了对果实的识别。

图 4-5 猕猴桃果实图片

```
ultralytics > test.py > ...
from ultralytics import YOLO

yolo = YOLO("./runs/detect/train5/weights/best.pt", task="detect")

result = yolo(source="./assests/kiwi1.png")
```

```
image 1/1 C:\Users\32347\Desktop\ultralytics-8.1.0\assests\kiwi1.png: 640x512 1 Kiwifruit, 209.0ms
Speed: 15.8ms preprocess, 209.0ms inference, 0.0ms postprocess per image at shape (1, 3, 640, 512)
```

图 4-6 模型部署与识别

最终在进行模型预测实验后，该模型已经可以投入应用，但本文所进行的模型预测仅停留在目标识别，还可以结合 matplotlib 对识别结果进行更深入的分析，得到详细参数上的可视化结果，由于作者水平与精力有限，暂时无法完成，在此不过多赘述。

第 5 章 人机交互软件设计

在基于 YOLOv8 的猕猴桃识别系统中，人机交互（UI）软件界面是用户与系统交互的桥梁，它直接影响用户体验和系统的易用性。本章详细介绍了使用 Python 软件编写的 UI 程序，该程序能够实现对输入图片的检测，并识别出图片中是否存在猕猴桃果实，最终标注出猕猴桃果实的位置。

5.1 UI 界面设计

设计一个 UI 界面，要完成的任务是能够对输入的图片进行检测，若有猕猴桃果实则标注出果实位置，据此可以规划出大致的界面布局，一个图片框用于图片输入与识别结果输出，两个按钮分别用于输入图片和目标检测，到此基本功能就能得到实现；但作者在程序中加入了“机化 2201 黄钊伟 2022013106”的个人信息作为防伪标识，最终编写的程序部分内容如下图，完整代码将在附录中列出。

UI.py > ...

```python
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageDraw, ImageFont, ImageTk
from ultralytics import YOLO


def detect_objects():
    results = model(selected_image_path.get())

    original_image = Image.open(selected_image_path.get())
    draw = ImageDraw.Draw(original_image)

    class_mapping = {0: 'kiwifruit', 1: 'Non'}

    font = ImageFont.truetype("arial.ttf", 15)

    conf, cls_idx = results[0].boxes.data.tolist()[0][-2:]
    boxes = results[0].boxes.xyxy.tolist()
    for box in boxes:
        x1, y1, x2, y2 = box

        draw.rectangle([x1, y1, x2, y2], outline='yellow', width=2)

        label = f'{class_mapping[int(cls_idx)]} {conf:.2f}'
        draw.text((x1, y1), label, fill='blue', font=font)
```

图 5-1 UI 界面设计程序

5.2 APP 测试

在 4.1 中完成 UI 界面设计后，运行效果如图 5-2 所示。

图 5-2 APP 界面

APP 成功运行后，开始进行测试，这时第三章中准备的验证用数据集派上了用场，插入的图片为未标注的原始图片，插入后的识别效果如下。

图 5-3 近景图片识别

从图 5-3 中可以看出，插入的图片中共有三颗猕猴桃果实，并且成功识别出来了，并且识别结果的“kiwifruit 0.94”中的 0.94 表示识别结果的置信度较高，在实际应用时也可以完成果实识别，因此可以得出结论该 APP 可以准确识别出近景图片中的猕猴桃果实。

图 5-4 中距离图像识别

除了近景图片，实际生产中还会存在中远距离检测是否有果实以及定位果实的情况，基于此，对 APP 进行多次实验，实验图片包括中距离图像及远距离图像，实验结果如图 5-4 与图 5-5 所示。

图 5-5 远距离图像识别

从实验结果中可以得出结论，APP 在中距离图片的识别上效果相比近景图片已有不小差距，在图片中也只能实现中短距离的果实识别，距离远了后如图 5-5 所示，由于多数果实距离较远还存在遮挡现象，因此识别效果很不理想，这也是未来系统的改进方向之一，即优化数据集，要扩大数据集体量，将各种生产情况考虑在内，确保不论什么角度的果实都可以成功识别，避免因错漏导致的果实浪费与生产率的降低。

第 6 章 总结展望

6.1 总结

本文成功设计并实现了基于 YOLOv8 算法的猕猴桃果实识别系统，该系统以其高效、准确的目标检测能力在图像识别领域展现出显著的应用潜力。系统的设计和实现涵盖了环境配置、数据集准备、模型训练与优化、实验验证等多个关键技术环节，为本人的大创项目《猕猴桃采摘机械臂设计》提供了视觉识别模块的设计思路与基础。

本文所设计的基于 YOLOv8 算法的猕猴桃果实识别系统存在以下优点：

（1）准确性：YOLOv8 算法以其快速、准确的检测能力而受到广泛关注，本文中的系统在验证集上的识别成功率达到了 94%以上，有较高的识别准确率。

（2）实时性：系统能够在极短的时间内完成图像的处理与识别，满足了实时检测的需求，对采摘机械臂的快速响应至关重要。

（3）易用性：设计的 UI 界面简洁直观，通过简单的操作完成图片的上传和结果的查看，降低了使用门槛。

（4）可扩展性：系统设计考虑了未来的功能扩展，如增加新的检测类别、优化检测算法等，改进起来较为方便。

但同时也存在着以下几点不足之处：

（1）复杂环境的工作效率：尽管系统在验证集上表现良好，但在复杂环境下，如果实重叠或光照变化剧烈时，识别准确率可能会下降。

（2）数据集依赖性：系统的识别性能在很大程度上依赖于训练数据集的质量和多样性，本文在训练模型时使用的数据集存在图片单一化的问题，清一色是单个的猕猴桃果实或者少量猕猴桃果实，对中远距离图片的训练有所欠缺，这也是目前系统只能应用在近距离果实识别的主要原因所在。

（3）动态目标跟踪能力有限：当前系统主要针对静态图像进行识别，对于移动目标的实时跟踪能力尚未涉及，这在机械臂动态采摘中是一个重要考虑因素。

6.2 展望

本文所设计的猕猴桃果实识别系统为未来作者的大创项目《猕猴桃采摘机械臂设计》提供了很好的视觉模块基础，应用前景广阔。

总体来说未来要应用到机械臂上实现集成化、一体化还需在以下几个方面做出努力：

（1）移动目标的实时检测：未来的研究可以将该系统迁移到移动平台，实现对动态目标的实时检测。这需要系统能够快速适应目标的运动变化，并提供稳定的识别结果。

（2）数据集扩充与泛化能力提升：通过扩充训练数据集，增加更多场景和条件下的图像样本，提高系统的泛化能力。同时，可以探索无监督或半监督学习方法，减少对标

注数据的依赖。

（3）集成到机械臂：将识别系统集成到猕猴桃采摘机械臂中，实现从识别到采摘的全自动化流程。这需要系统与机械臂的控制系统紧密配合，实现精确的定位和操作。

（4）交互优化：进一步优化 UI 界面，提供更丰富的交互功能和更直观的反馈，提升用户的使用体验。例如，增加实时视频流的识别展示，让用户能够实时监控机械臂的工作状态。

我相信，YOLOv8 在猕猴桃生产领域所能做出的贡献绝不仅限于此，但受限于作者水平，本文仅作为抛砖引玉的基础，希望能为未来猕猴桃生产的进一步研究提供一定的参考价值，但这不会是终点，而是新的起点，也希望本人在两年后、五年后、十年后再次点开这篇论文能够回想起来原来我在图像处理领域的开始是设计了这样一个简单但完整的猕猴桃果实识别系统。

致谢

岁月不居，时节如流，一转眼一个学期的时间就过去了，虽然图像处理与机器视觉这门课只上了九个星期的课，但我仍旧受益匪浅。

首先感谢赵老师，在上课期间一直为我们讲解相关知识，也传授给我们过来人的经验。从功利性的角度出发，我一开始选课的原因是从学长学姐那了解到赵老师为人非常亲和友好，而且给分很高。但是通过大半个学期的学习，我逐渐从应对作业到真正对数字图像处理这个领域产生了兴趣。虽然我之前参加过工创赛和机器人竞赛，在视觉方面有所涉猎，但也仅限于完成任务，没有过多的思考。在这学期的课程学习中，我通过一次次的课堂实战以及作业巩固，慢慢养成了对图像处理相关问题进行思考的习惯，因此要感谢赵老师在图像处理点燃了我的兴趣。

我深知由于课时数有限，我们所接触到的图像处理知识只是皮毛。但我相信由衷的兴趣可抵百十小时的按图索骥，以兴趣为老师，未来一定会越学越投入。也正是因此，才有了我这次结课作业的选题，为此我学习了很多 YOLOv8 的相关背景还有 Python 的 UI 界面设计方法，虽然不简单，但我能乐在其中。

最后再次感谢赵老师，从图像处理这门课中我对一个新的领域产生了兴趣，也从中获得了开展大创项目设计视觉模块的新思路。从课程中学到知识，并且能现学现卖应用在自己需要的方面，我想这就是学习的乐趣所在吧。

参考文献

[1]朱拴祥,祝晨.猕猴桃生产全程机械化让农业壮起来[J].中国农民合作社,2024,(08):49-50.

[2]朱拴祥,薛云飞.对猕猴桃生产全程机械化的思考[J].西北园艺(果树),2020,(06):14-17.

[3]汪小霞,张莹,叶可辉.陕西省有机猕猴桃生产技术研究[J].陕西农业科学,2016,62(11):94-98.

[4]陕西省周至县成为全球最大猕猴桃生产基地县[J].农村百事通,2015,(09):13.

[5]慕军营,陈军,孙高杰,等. 基于机器视觉的猕猴桃特征参数提取 [J]. 农机化研究, 2014, 36 (06): 138-142. DOI:10.13427/j.cnki.njyi.2014.06.035.

附　录

附录 A 本文所涉及的网站链接

1、miniconda 下载

miniconda: https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/

2、PyPI 下载

pypi: https://mirrors.tuna.tsinghua.edu.cn/help/pypi/

3、Pytorch 下载

pytorch: https://pytorch.org/

4、Ultralytics 下载

ultralytics: https://github.com/ultralytics/ultralytics

附录 B 模型训练相关程序

1、模型训练参数

```
# Ultralytics YOLO , AGPL-3.0 license
# COCO8 dataset (first 8 images from COCO train2017) by Ultralytics
# Documentation: https://docs.ultralytics.com/datasets/detect/coco8/
# Example usage: yolo train data=coco8.yaml
# parent
# ├── ultralytics
# └── datasets
#       └── coco8  ← downloads here (1 MB)

# Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]
path: kiwi  # dataset root dir
train: images/train  # train images (relative to 'path') 4 images
val: images/val  # val images (relative to 'path') 4 images
test:  # test images (optional)

# Classes
names:
  0: Kiwifruit
```

2、模型训练程序

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')

model.train(data='yolo-kiwi.yaml',workers=1,epochs=200,batch=16)
```

3、模型验证程序

```
from ultralytics import YOLO

if __name__ == "__main__":
    # Load a model
    pth_path = r"C:\Users\32347\Desktop\ultralytics-8.1.0\runs\detect\train5\weights\best.pt"
    # model = YOLO('yolov8n.pt')   # load an official model
    model = YOLO('yolov8l.pt')   # load a custom model

    # Validate the model
    metrics = model.val()   # no arguments needed, dataset and settings remembered
    metrics.box.map   # map50-95
    metrics.box.map50   # map50
    metrics.box.map75   # map75
    metrics.box.maps   # a list contains map50-95 of each category
```

4、模型部署测试程序

```
from ultralytics import YOLO

yolo = YOLO("./runs/detect/train5/weights/best.pt", task="detect")

result = yolo(source="./assests/kiwi1.png")
```

附录 C 人机交互 UI 程序

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageDraw, ImageFont, ImageTk
from ultralytics import YOLO

def detect_objects():
    results = model(selected_image_path.get())
    original_image = Image.open(selected_image_path.get())
    draw = ImageDraw.Draw(original_image)
    class_mapping = {0: 'kiwifruit', 1: 'Non'}
    font = ImageFont.truetype("arial.ttf", 15)
    conf, cls_idx = results[0].boxes.data.tolist()[0][-2:]
    boxes = results[0].boxes.xyxy.tolist()
    for box in boxes:
        x1, y1, x2, y2 = box
        draw.rectangle([x1, y1, x2, y2], outline='yellow', width=2)
        label = f'{class_mapping[int(cls_idx)]} {conf:.2f}'
```

```
            draw.text((x1, y1), label, fill='blue', font=font)
        original_image = original_image.resize((400, 400))
        tk_image = ImageTk.PhotoImage(original_image)
        selected_image_label.configure(image=tk_image)
        selected_image_label.image = tk_image
    def select_image():
        file_path = filedialog.askopenfilename(title="选择一张图片", filetypes=[("图片文件",
"*.jpg;*.png;*.jpeg;*.bmp")])
        if file_path:
            selected_image_path.set(file_path)
            # 显示选定的图片
            selected_image = Image.open(file_path)
            selected_image = selected_image.resize((400, 400))
            tk_image = ImageTk.PhotoImage(selected_image)
            selected_image_label.configure(image=tk_image)
            selected_image_label.image = tk_image
    root = tk.Tk()
    root.title("猕猴桃果实识别系统")
    root.geometry("550x600")
    selected_image_path = tk.StringVar()
    model = YOLO(r'C:\Users\32347\Desktop\ultralytics-8.1.0\runs\detect\train5\weights\best.pt')
    select_button = tk.Button(root, text="选择图片", command=select_image, width=15, height=2)
    select_button.place(x=80, y=480)
    label_name = tk.Label(root, text="机化2201 黄钊伟 2022013106", font=("Arial", 15))
    label_name.pack(pady=20)
    label_name.place(x=120, y=20)

    detect_button = tk.Button(root, text="检测目标", command=detect_objects, width=15, height=2)
    detect_button.place(x=340, y=480)
    selected_image_label = tk.Label(root)
    selected_image_label.place(x=70, y=60)
    image_label = tk.Label(root)
    image_label.place(x=500, y=60)
    root.mainloop()
```